

LC 1298: Maximum Candies You Can Get from Boxes

Authored by: Arpit Raj, LNMIIT Jaipur | July 27, 2026

Problem Variables & Data Structures

Given n boxes labeled from 0 to $n - 1$, and the following 4 arrays:

status[i]

1 if open, 0 if closed.

keys[i]

List of labels of boxes you can open later by opening the i -th box.

candies[i]

Number of candies inside the i -th box.

containedBoxes[i]

List of boxes found inside the i -th box.

initialBoxes

List of labels of boxes you initially have.

Goal: We need the maximum number of candies.

Observations & Logic

- **Observation:** A BOX can only be opened if you own it AND you have its key.
- We will maintain states: have box, have key, and opened.

Queue Operations

Queue stores the box number to process. If a box is already open, simply process it. Processing a box means:

- Pop one box from the queue.
- Collect its candies.
- Collect all its keys.
- Collect all the boxes inside it.
- If collecting a key allows you to open a box you already have, process it.
- If collecting a new box is already open (or you already have its key), process it.
- *In short:* If you already own the corresponding box of that key, push to queue.
VICE versa for box → key.

C++ Implementation

```
class Solution {
public:
    int maxCandies(vector<int>& status, vector<int>& candies,
vector<vector<int>>& keys, vector<vector<int>>& containedBoxes,
vector<int>& initialBoxes) {

        int n = status.size();
        vector<int> havebox(n, 0);
        vector<int> seen(n, 0);

        queue<int> q; // isme box rkhenhe to be processed

        for(auto &box : initialBoxes) {
            havebox[box] = 1; // ho skta iski chabhi baad me mile
            if(status[box] == 1) {
                q.push(box);
                seen[box] = 1;
            }
        }

        int ans = 0;
        while(!q.empty()) {
            int currbox = q.front();
            q.pop();

            ans += candies[currbox];

            for(auto &key : keys[currbox]) {
                status[key] = 1;

                if(havebox[key] && !seen[key]) {
                    q.push(key); // key ke index wala box push kr
                    seen[key] = 1;
                }
            }

            for(auto &child : containedBoxes[currbox]) {
                havebox[child] = 1;
                if(status[child] == 1 && !seen[child]) {
                    q.push(child);
                    seen[child] = 1;
                }
            }
        }
    }
};
```

```
        }  
    }  
}  
  
    return ans;  
}  
};
```